

Documentation de l'application web

1 tonne *de* bonnes pratiques *de* **green IT**

par Thibaut Chantrel, Lou Delcourt, Jade Le Roux,
Grégoire Muller, Meije Pigeonnat, Sarah Pignol

Mis à jour en mai 2026 par
Elie Ravoux, Nathan Aknin, Thibault Gallapont,
Lisa Quantin

Table des matières

1. Architecture et technologies.....	2
a. NestJS avec Websocket socket.io.....	2
b. React pour le Frontend.....	2
c. PostgreSQL pour la Base de Données.....	2
2. Espace de travail :.....	3
a. API (Backend).....	3
b. Front (Frontend).....	4
c. Shared.....	5
3. Les configurations.....	5
a. BD :.....	5
b. Structure fichier .env :.....	5
c. Démarrage :.....	6
4. Documentations des services back-end.....	6
a. Users.....	6
b. Card.....	6
c. Authentification.....	6
d. Sensibilisation.....	7
e. Game.instance.....	7
f. Game.gateway.....	8
g. Booklet.....	9
5. Spécifications.....	10
a. Spécifications du back-end.....	10
b. Spécifications du front-end.....	10

1. Architecture et technologies

a. NestJS avec Websocket socket.io

NestJS est un framework Node.js progressif pour construire des applications server-side efficaces, fiables et évolutives. Il utilise le langage TypeScript et est basé sur les principes de Angular. Nous avons choisi NestJS pour la construction du backend de notre jeu en raison de sa facilité de configuration, de son architecture modulaire et de son intégration transparente avec d'autres bibliothèques et frameworks.

Nous avons utilisé Websocket socket.io, une bibliothèque JavaScript pour les applications web temps réel, pour la communication en temps réel entre le client et le serveur. Cette technologie permet une communication bidirectionnelle entre le client et le serveur, ce qui est essentiel pour un jeu multijoueur interactif.

b. React pour le Frontend

React est une bibliothèque JavaScript open-source pour la construction d'interfaces utilisateur. Il est maintenu par Facebook et une communauté active de développeurs. Nous avons choisi React pour le développement du frontend de notre jeu en raison de sa facilité de création d'interfaces utilisateur dynamiques et interactives, ainsi que de sa gestion efficace de l'état de l'application.

Ajout de la bibliothèque **Axios** pour centraliser les appels API.

- Tous les appels `fetch()` ont été remplacés par des appels via l'instance `api` d'Axios.
- Le routeur (`App.tsx`) supporte désormais un préfixe dynamique via `basename={import.meta.env.VITE_APP_PREFIX || '/'}`.

c. PostgreSQL pour la Base de Données

PostgreSQL est un système de gestion de base de données relationnelle open-source et puissant. Nous avons utilisé PostgreSQL comme base de données pour stocker les données du jeu, y compris les informations sur les joueurs, les scores, les parties en cours, etc. PostgreSQL offre une grande robustesse, une grande fiabilité et une compatibilité élevée avec les normes SQL, ce qui en fait un choix idéal pour les applications nécessitant une gestion efficace des données.

d. NGINX pour le reverse proxy

NGINX est un serveur web et un reverse proxy haute performance open-source. Nous avons utilisé NGINX comme passerelle interne, déployée via Docker, pour gérer et rediriger le flux de données entre les clients et notre API NestJS. NGINX offre une grande stabilité, une configuration flexible pour les WebSockets et une gestion efficace des charges de trafic, ce qui en fait un choix idéal pour sécuriser les points d'entrée et assurer la liaison fluide entre les différents conteneurs de notre architecture.

Activation du fichier `nginx.conf` dans le Dockerfile Frontend.

- Le serveur écoute sur le port **8083**.
- Configuration de la redirection des appels `/gameNR/api/` vers le backend et `/gameNR/ws/` pour les WebSockets.

e. Docker pour la conteneurisation

Docker est une plateforme open-source permettant de déployer des applications dans des conteneurs isolés. Nous avons utilisé Docker pour encapsuler notre API, notre serveur NGINX et notre base de données dans des environnements cohérents et reproductibles. Docker offre une grande facilité de déploiement, une isolation complète des ressources et une gestion simplifiée des dépendances, ce qui en fait un choix idéal pour garantir que l'application fonctionne de manière identique sur toutes les machines de développement et de production.

Nouvelle architecture multi-composition

- `docker-compose-bd-only.yml` : Pour lancer uniquement la base de données.
- `docker-compose-dev.yml` : Nouvelle version pour le développement.
- `docker-compose-prod.yml` : Version de production (sans base de données locale).
- Les conteneurs sont désormais nommés explicitement : `gameNR_api`, `gameNR_front`, `gameNR_db`.

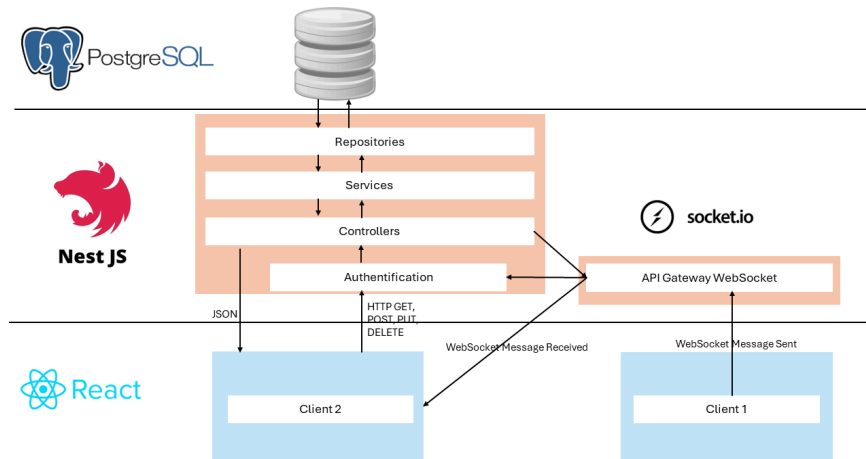


Schéma de l'architecture technique sans docker

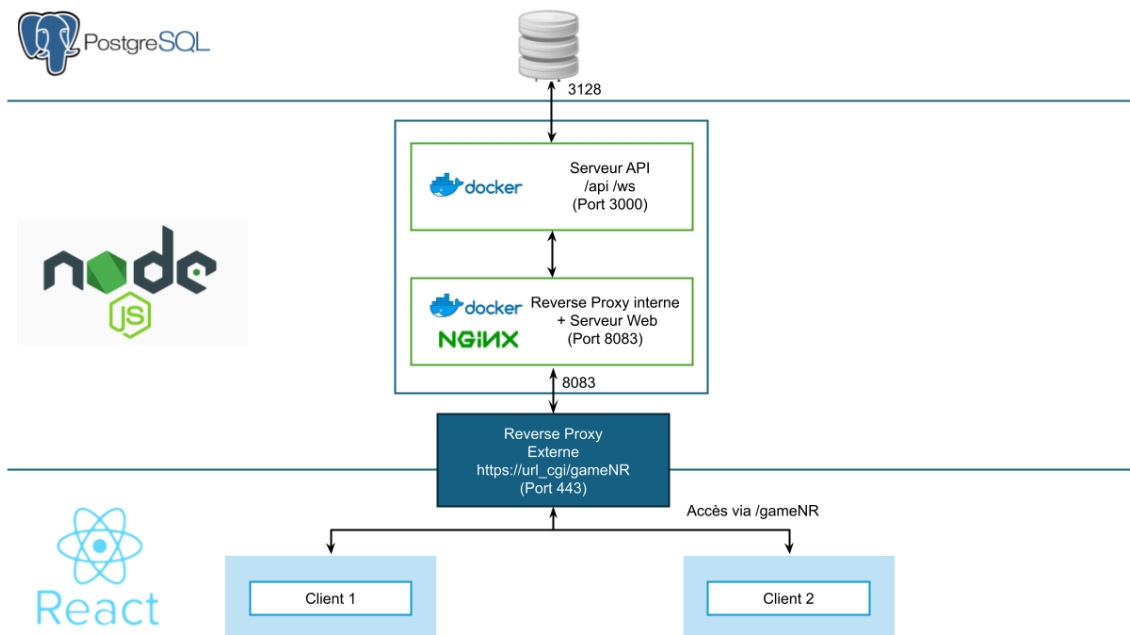


Schéma de l'architecture technique de production

2. Espace de travail :

Chaque espace de travail est organisé de manière à faciliter le développement, la maintenance et le déploiement du jeu.

a. API (Backend)

L'espace de travail `api` contient le code backend développé avec NestJS. Voici l'organisation générale du code dans cet espace de travail :

- **src/authentication** : Ce répertoire contient les fichiers relatifs à l'authentification des utilisateurs, tels que les stratégies d'authentification, les middleware d'authentification, etc.
- **src/booklet** : Ce répertoire contient les fichiers concernant la gestion des livrets de jeu, qui peuvent contenir des informations sur les règles du jeu, les objectifs, les succès, etc.
- **src/card** : Ce répertoire contient les fichiers liés à la gestion des cartes utilisées dans le jeu. Cela peut inclure la logique pour créer, distribuer et gérer les cartes.
- **src/entity** : Ce répertoire contient les entités de base de données qui représentent les objets principaux de l'application, tels que les utilisateurs, les jeux, les cartes, etc.
- **src/game** : Ce répertoire contient les fichiers liés à la logique métier du jeu. On y trouve la logique pour démarrer, arrêter, mettre en pause, reprendre le jeu, ainsi que pour gérer les tours, les phases, etc.
- **src/sensibilisation** : Ce répertoire contient les fichiers liés à la sensibilisation, tels que les questions de sensibilisation.
- **src/users** : Ce répertoire contient les fichiers concernant la gestion des utilisateurs, tels que les opérations CRUD (création, lecture, mise à jour, suppression) sur les utilisateurs etc.
- **src/websocket** : Ce répertoire contient les fichiers relatifs à la gestion des Websockets pour la communication en temps réel avec les clients. Cela peut inclure les gestionnaires d'événements, les middlewares, etc.

Cette structure permet d'organiser le code de manière logique et cohérente, en regroupant les fonctionnalités connexes dans des répertoires distincts. Cela

facilite la navigation dans le code et la collaboration entre les membres de l'équipe de développement.

b. Front (Frontend)

L'espace de travail ``front`` contient le code frontend développé avec React. Voici l'organisation générale du code dans cet espace de travail :

- **src/components** : Ce répertoire contient les composants réutilisables de notre application React. Les composants peuvent être des éléments d'interface utilisateur tels que des boutons, des formulaires, des cartes, etc.
- **src/pages** : Ce répertoire contient les composants de page de l'application. Chaque fichier peut représenter une page distincte de l'application, par exemple la page d'accueil, la page de profil utilisateur, la page de jeu, etc.
- **src/js/components** : Ce répertoire contient les composants spécifiques TypeScript et CSS qui sont utilisés dans l'application.
- **src/js/hooks** : Ce répertoire contient les hooks personnalisés que l'on a créés pour réutiliser la logique dans nos composants fonctionnels.
- **src/js/pages** : Ce répertoire contient les pages de l'application. Ces composants peuvent être des classes ou des fonctions et sont utilisés pour afficher les différentes pages de l'application.

c. Shared

L'espace de travail ``shared`` contient le code partagé entre le backend et le frontend. Voici l'organisation générale du code dans cet espace de travail :

- **shared/common** : Ce répertoire contient les éléments communs utilisés à la fois par le client et le serveur. Cela inclut des constantes, des types de données partagés, des utilitaires, des fonctions de validation, etc.
- **shared/client** : Ce répertoire contient le code spécifique au client partagé entre les différentes parties de l'application. Cela inclut des événements clients, des fonctions de gestion des erreurs, des interfaces utilisateur partagées, etc.
- **shared/server** : Ce répertoire contient le code spécifique au serveur partagé entre les différentes parties de l'application. Cela inclut des middlewares, des utilitaires de validation côté serveur, des modèles de données, etc.

3. Les configurations

a. BD :

Lors de la première installation, il faut initialiser la base de données grâce à une requête.

Pour initialiser une BD vide, deux fichiers .csv sont à disposition pour respectivement charger les cartes à jouer et les quizz de sensibilisation.

Voici deux exemples de requête sur Postman :

The image shows two screenshots of the Postman interface. Both are for POST requests.

Top Screenshot: The URL is `http://localhost:9000/sensibilisation/csv`. The 'Body' tab is selected, and 'form-data' is chosen. A table shows a key `csvFile` with a file value `sensibilisation.csv`.

Key	Value	Description
csvFile	sensibilisation.csv	

Bottom Screenshot: The URL is `http://localhost:9000/card/csv`. The 'Body' tab is selected, and 'form-data' is chosen. A table shows a key `csvFile` with a file value `Liste-cartes-avec-descripti...`.

Key	Value	Description
csvFile	Liste-cartes-avec-descripti...	

b. Structure fichier .env :

```
DATABASE_USER = <data_base_name>
DATABASE_PASSWORD = <password_data_base>
DATABASE_HOST = localhost
DATABASE_PORT = 5432
DATABASE_URL = CGI-AGIR-INSA
CORS_ALLOW_ORIGIN = http://localhost:5173
```

```
VITE_API_URL = http://localhost:8083
VITE_APP_PREFIX=/gameNR
```


c. Démarrage :

Depuis le terminal, dans l'espace suivant \smartcgi :

- pour run le client : `npm run client`
- pour run le serveur : `npm run server`

Pour lancer via Docker avec la nouvelle configuration : `docker-compose -f docker-compose-dev.yml up` L'application sera accessible sur : <http://localhost:8083/gameNR>

4. Documentations des services back-end

a. Users

Fonctionnalité	Description	Input	Output
createUser	Créer un utilisateur dans la base de données avec ses attributs	mail: string, password: string, lastname: string, firstname: string	user_id : number
findOne	Trouve un utilisateur dans la base de données à partir de son mail	mail: string	user : User

b. Card

Fonctionnalité	Description	Input	Output
parseCsv	Créer les cartes dans la base de données à partir d'un fichier csv	file: Express.Multer.File	
getDeck	Créer la pioche initiale de 97 cartes mélangées		cards : Card[]

c. Authentification

Fonctionnalité	Description	Input	Output
signUp	Inscrit un utilisateur, appelle le service createUser	mail: string, password: string, lastname: string, firstname: string	success: boolean
signIn	Permet la connexion d'un utilisateur	mail: string, password: string	token: string
signOut	Permet la déconnexion d'un utilisateur	token: string	success: boolean
isConnected	Permet de tester si un user est actuellement connecté	token: string	success: boolean

d. Sensibilisation

Fonctionnalité	Description	Input	Output
parseCsv	Analyse un fichier CSV pour extraire des données de questions	Fichier CSV	Cartes de questions
randomQuestionToStart	Sélectionne aléatoirement une question pour démarrer le jeu	Aucun	Question sélectionnée
getSensibilisationQuizz	Récupère une question de sensibilisation pour le quizz	Aucun	Question de quizz
getGoodSolution	Récupère la bonne solution pour une question donnée	ID de question	Bonne solution

e. Game.instance

Fonctionnalité	Description	Input	Output
triggerStart	Démarre une partie en initialisant les decks de cartes et les joueurs	Aucun	Aucun
triggerFinish	Termine la partie et génère un rapport de jeu	ID du gagnant, nom du gagnant	Aucun
discardCard	Jette une carte de la main du joueur	Carte, Socket authentifié	Aucun
playCard	Joue une carte depuis la main du joueur	Carte, Socket authentifié	Aucun
answerBestPracticeQuestion	Enregistre la réponse à une question de meilleure pratique	ID du joueur, ID de carte, réponse	Aucun
answerBadPracticeQuestion	Enregistre la réponse à une question de mauvaise pratique	ID du joueur, ID de carte, réponse	Aucun
answerSensibilisationQuestion	Enregistre la réponse à une question de sensibilisation	ID du joueur, réponse	Aucun
discardCard	Jette une carte de la main du joueur	Carte, Socket authentifié	Aucun
playCard	Joue une carte depuis la main du joueur	Carte, Socket authentifié	Aucun
answerBestPracticeQuestion	Enregistre la réponse à une question de meilleure pratique	ID du joueur, ID de carte, réponse	Aucun
answerBadPracticeQuestion	Enregistre la réponse à une question de mauvaise pratique	ID du joueur, ID de carte, réponse	Aucun
answerSensibilisationQuestion	Enregistre la réponse à une question de sensibilisation	ID du joueur, réponse	Aucun

f. Game.gateway

Fonctionnalité	Description	Input	Output
onLobbyCreate	Gère la création d'un lobby	Client authentifié, données de création	Aucun
onLobbyJoin	Gère la connexion à un lobby	Client authentifié, données de connexion	Aucun
onLobbyLeave	Gère la déconnexion d'un lobby	Client authentifié	Aucun
onLobbyStartGame	Gère le démarrage d'une partie dans un lobby	Client authentifié, données de démarrage	Aucun
onClientReconnect	Gère la reconnexion d'un client	Client authentifié, données de reconnexion	Aucun
onPlayCard	Gère le jeu d'une carte dans une partie	Client authentifié, données de jeu	Aucun
onDiscardCard	Gère le jet d'une carte dans une partie	Client authentifié, données de jet	Aucun
onPracticeQuestion	Gère la réponse à une question de pratique	Client authentifié, données de réponse	Aucun
onSensibilisationQuestion	Gère la réponse à une question de sensibilisation	Client authentifié, données de réponse	Aucun
onSensibilisationQuestionGet	Gère la demande de question de sensibilisation	Client authentifié	Aucun

g. Booklet

Fonctionnalité	Description	Input	Output
createBooklet	Crée un nouveau livret Green IT pour un utilisateur	ID de l'utilisateur	Livret Green IT créé
getBooklet	Récupère le livret Green IT d'un utilisateur	ID de l'utilisateur	Livret Green IT récupéré

5. Spécifications

a. Spécifications du back-end

Utilisation de jetons JWT : Le back-end utilise des jetons JWT (JSON Web Tokens) pour gérer l'authentification et l'autorisation des utilisateurs. Ces jetons sont générés lors de la connexion réussie d'un utilisateur et sont utilisés pour authentifier les requêtes ultérieures.

Hachage des mots de passe : Les mots de passe des utilisateurs sont hachés avant d'être stockés dans la base de données. Le hachage est effectué à l'aide de la bibliothèque bcrypt, ce qui garantit la sécurité des mots de passe en les rendant difficilement réversibles.

Gestion des sockets :

b. Spécifications du front-end

L'application supporte désormais un déploiement sous un sous-répertoire (ex: `/gameNR`).

- La configuration de la base d'URL dans Vite (`vite.config.ts`) est dynamique :
`base: process.env.VITE_APP_PREFIX || '/'`.
- Utilisation systématique d'Axios pour la communication API afin de gérer proprement les préfixes d'URL et les en-têtes.